Rapport projets deep learning et data

Cyrian Revel–Hidalgo

27 octobre 2026

Sommaire

I Traitement d'images satellites **2**

1 Introduction **2**

2 Modèles CNN **2**
2.1 Régression logistique . . . 2
2.2 Modèle simple . . . 3
2.3 Modèle profond . . . 4
2.4 Potentielles améliorations . . . 4

3 Transfert Learning et Fine-tuning **4**
3.1 EfficientNetB0 . . . 4
3.2 EfficientNetV2 Small . . . 5
3.3 MobileNetV3 . . . 6

4 Conclusion et Discussion **8**
4.1 Synthèse des performances . . . 8
4.2 Analyse de la généralisation sur l'ensemble de test . . . 8

II Projet de classification de textes **10**

5 Introduction **10**
5.1 Nettoyage . . . 10

6 Présentation des données et statistiques descriptives **11**
6.1 Statistiques descriptives . . . 11
6.2 Validation croisée . . . 12

7 Présentation des méthodes et premiers résultats **12**
7.1 Modèle bayésien . . . 12
7.2 Réseau dense . . . 13
7.3 LSTM . . . 14

8 Gestion des poids **15**

9 Conclusion et discussion **16**

Première partie

Traitement d'images satellites

1 Introduction

Le but du projet est d'utiliser différents modèles afin de faire de la classification d'images, selon la présence ou non d'une éolienne sur la photo. Nous possédons pour ce faire d'une base de données d'images satellites réparties dans 3 dossiers : Le jeu d'entraînement (train) dans lequel les images sont rangées dans "target" si elles contiennent une éolienne, dans "other" sinon ; un jeu de validation dont on connaît également la présence ou non d'une éolienne ; et un jeu de test, sur lequel on ne sait rien. L'objectif est d'entraîner un modèle et évaluer sa précision, pour ensuite prédire la classe des images test. Nos images sont de tailles 128 x 128 et en couleurs. Le jeu d'entraînement contient 58218 images, ceux de validations et de test en contiennent 5000 chacun. Afin de pouvoir faire mes propres tests, je récupère 20% des données de train afin d'en faire un jeu de test interne.

Pour ce projet j'utilise principalement les bibliothèques TensorFlow et Keras à partir d'un notebook Jupyter.

La première difficulté à laquelle j'ai dû faire face fut l'environnement de développement. En effet, je travaille sur une machine locale possédant une GTX 1070, ce qui n'est pas un GPU dernière génération, cela m'a donc obligé à passer sur une ancienne version de TensorFlow (TensorFlow 2.10) afin de pouvoir correctement utiliser la technologie CUDA (permettant de gagner du temps de calcul en entraînant les modèles sur le GPU plutôt que le CPU).

2 Modèles CNN

2.1 Régression logistique

Le premier modèle utilisé est un modèle de régression logistique, constitué de 49000 paramètres à entraîner, avec une fonction d'activation sigmoïde. Pour ce modèle et tout les suivants, la taille des batchs est de 32 et j'utilise de l'early stopping afin d'éviter le surapprentissage.

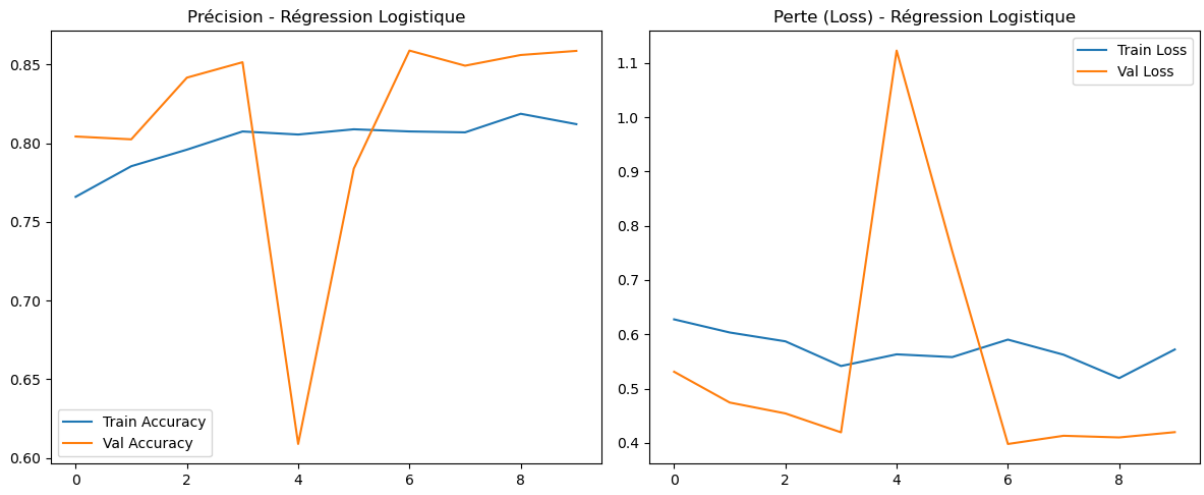


FIGURE 1 – Courbes d'apprentissage : Régression Logistique.

Ce premier modèle étant très simple, il est rapide à entraîner. L'entraînement s'arrête à la 10ème epoch avec une vitesse de 15s/epoch. On obtient avec celui-ci une précision de 85,84% sur le set de validation, et 85,87% sur celui de test interne.

2.2 Modèle simple

Le second modèle est un réseau de neurones convolutionnel assez simple, constitué de 7 000 000 de paramètres, une couche de convolution 2D avec un filtre 5x5 et la fonction ReLu, , du Max pooling, une couche dense avec la fonction reLu, et une sigmoïde en sortie.

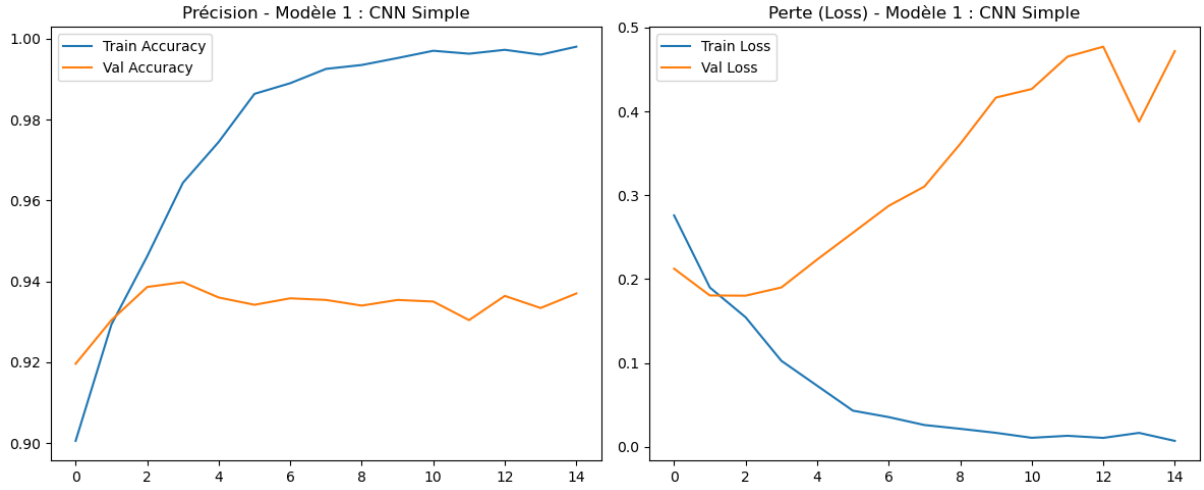


FIGURE 2 – Résultats et structure du modèle simple.

L'entraînement s'effectue en 15 epochs de 25s chacune. On obtient une précision de 93,34% sur les données de validation, et 94,10% sur celles de test interne.

2.3 Modèle profond

Améliorons ce modèle en rajoutant des couches de convulution de taille 128, puis 64 puis 32, avec du max pooling entre ces couches, et du dropout.

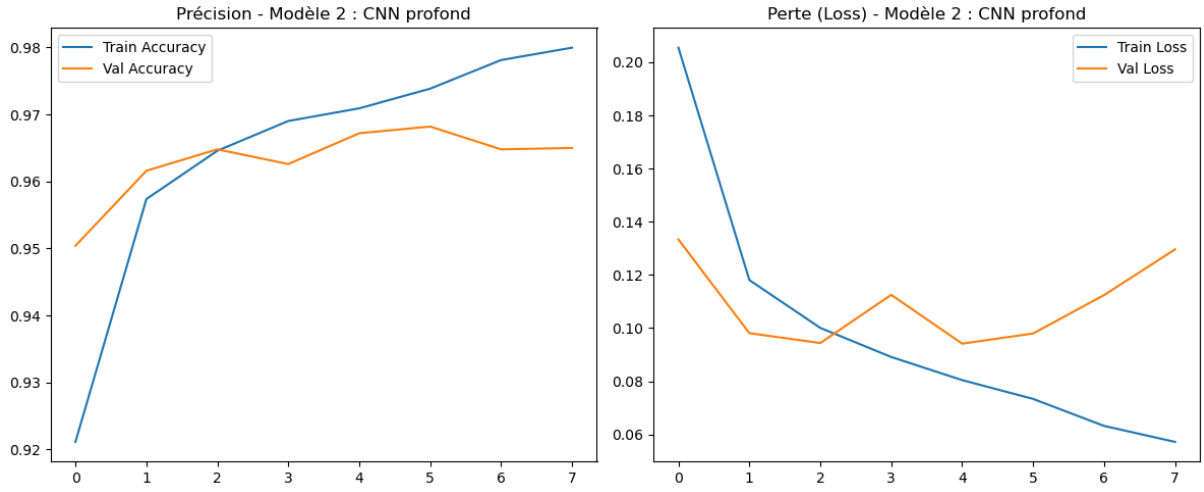


FIGURE 3 – Courbes de convergence du modèle profond.

Ce modèle contient 560 000 paramétres, L'entraînement nécessite 9 epochs de 75s. On obtient alors une précision de 96,86% sur le set de validation, et 97,28% sur celui de test interne.

2.4 Potentielles améliorations

J'ai essayé différentes améliorations : - Augmenter le jeu d'entraînement avec de la data augmentation : cela multiplie par 5 le temps d'entraînement mais n'a pas d'impact sur la précision des prédictions - Diminuer la taille des filtre en passant à une taille 3x3 : pas spécialement d'effet sur la précision, mais ralentit l'entraînement. - Passer la couche dense finale de 64 à 128 : augmente le nombre de paramétre et la durée des epochs, mais pas d'effet sur les prédictions - Le dropout permet une meilleure précision sur l'ensemble de test interne.

3 Transfert Learning et Fine-tuning

Nous allons maintenant opter consistant à réutiliser un modèle déjà entraîné, et à réentraîner les dernières couches de ce modèle sur nos données.

3.1 EfficientNetB0

Le premier modèle que nous allons utiliser est EfficientNetB0, un modèle de classification entraîné par Google. Celui-ci contient 4 000 000 de paramètres, mais nous n'entraînons qu'à partir de la 150e couche sur les 238 du modèle.

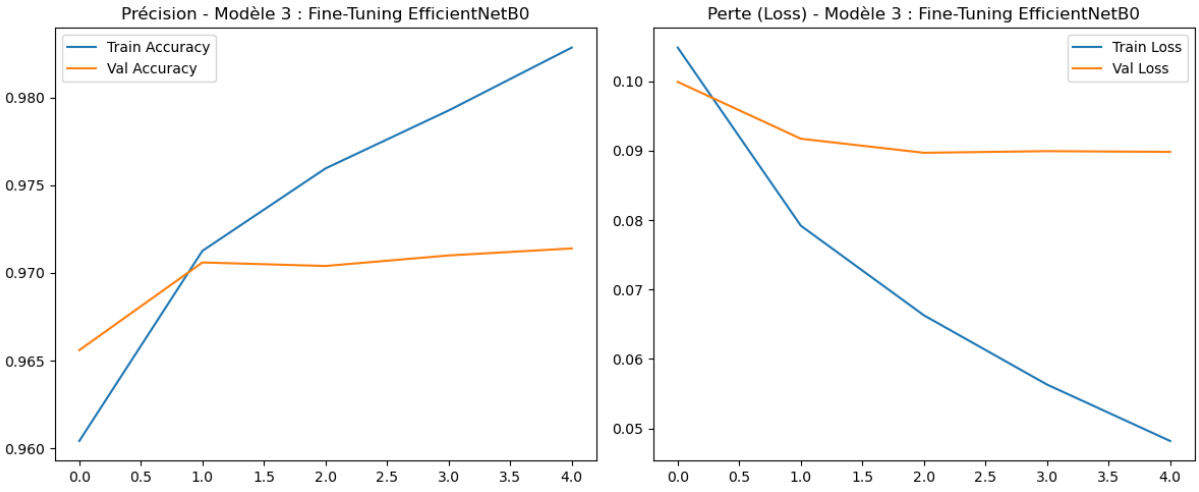


FIGURE 4 – Performance du Fine-tuning sur EfficientNetB0.

Après un entraînement de 9 epochs de 30s, puis 4 epochs de fine-tuning de 70s, on obtient 97,02% de bonnes prédictions sur l'ensemble de validation, et 97,38% sur celui de test interne.

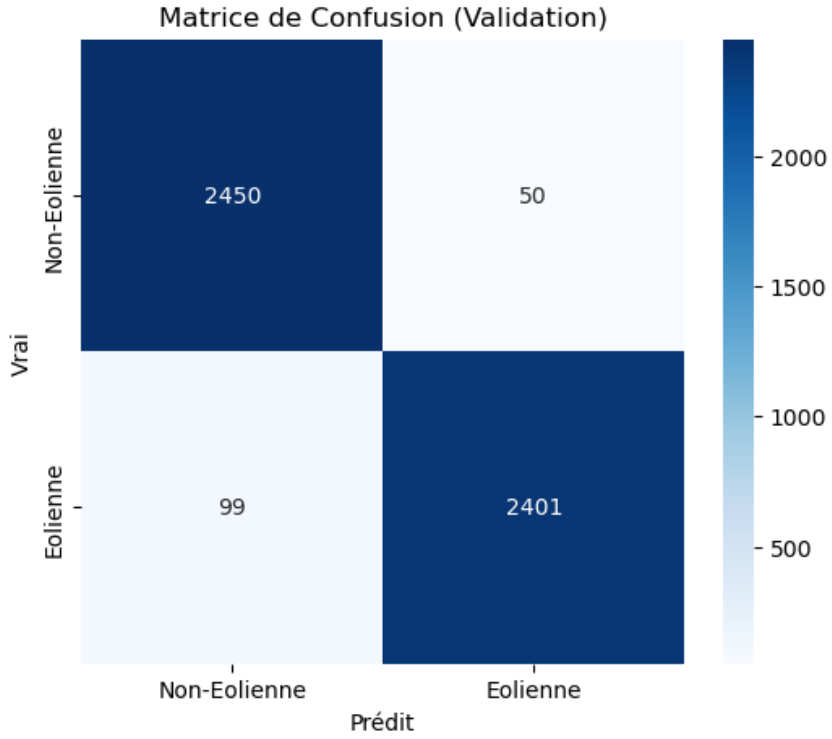


FIGURE 5 – Matrice de confusion : EfficientNetB0.

3.2 EfficientNetV2 Small

Le premier modèle que nous allons utiliser est EfficientNetV2, un modèle de classification également développé par Google. Celui-ci contient 20 000 000 de paramètres, mais nous n'entraînons qu'à partir de la 400e couche sur les 513 du modèle.

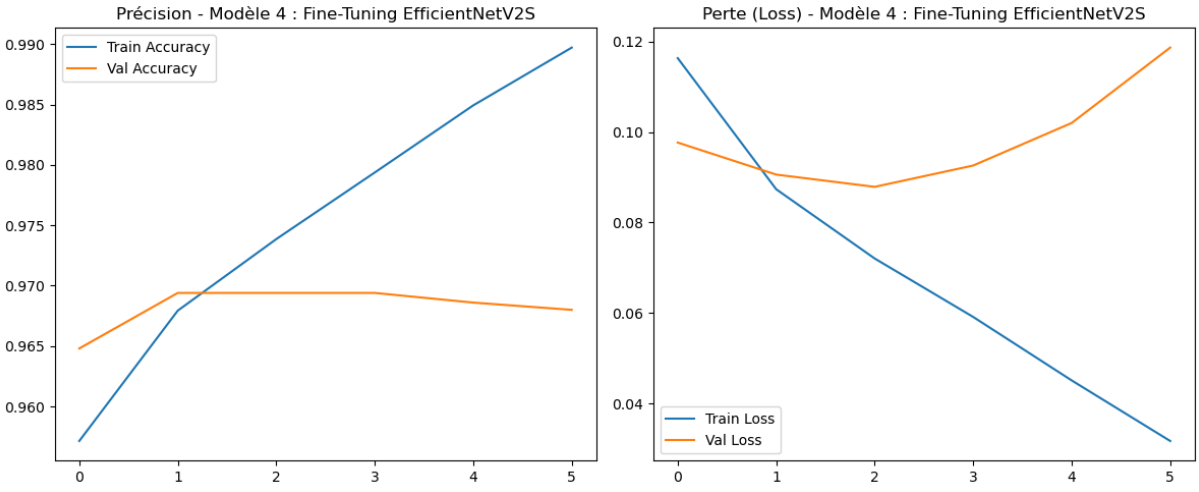


FIGURE 6 – Performance du Fine-tuning sur EfficientNetV2 Small.

L'entraînement s'effectue en 16 epochs de 90s chacunes, et 6 epochs de fine-tuning de 150s. On obtient une précision de 96,80% sur les données de validation, et 97,20% sur celles de test interne.

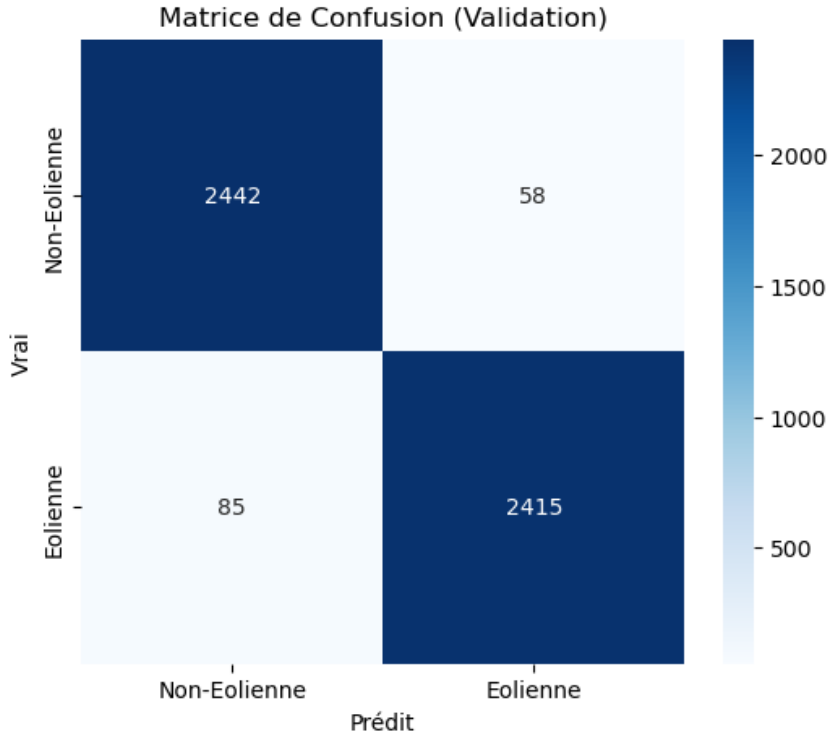


FIGURE 7 – Matrice de confusion : EfficientNetV2 Small.

3.3 MobileNetV3

Le dernier modèle que nous allons utiliser est MobileNetV3, un modèle que j'ai choisi car il a été conçu pour tourner sur mobile, il est donc plutôt léger. J'étais à la base parti sur le modèle ConvNext, mais celui-ci étant trop lourd, je n'ai pas réussi à l'utiliser,

même en réduisant fortement la taille des batchs. Il est constitué de seulement 3 000 000 de paramètres dans ses 263 couches. Nous effectuons du fine-tuning uniquement à partir de la 160e.

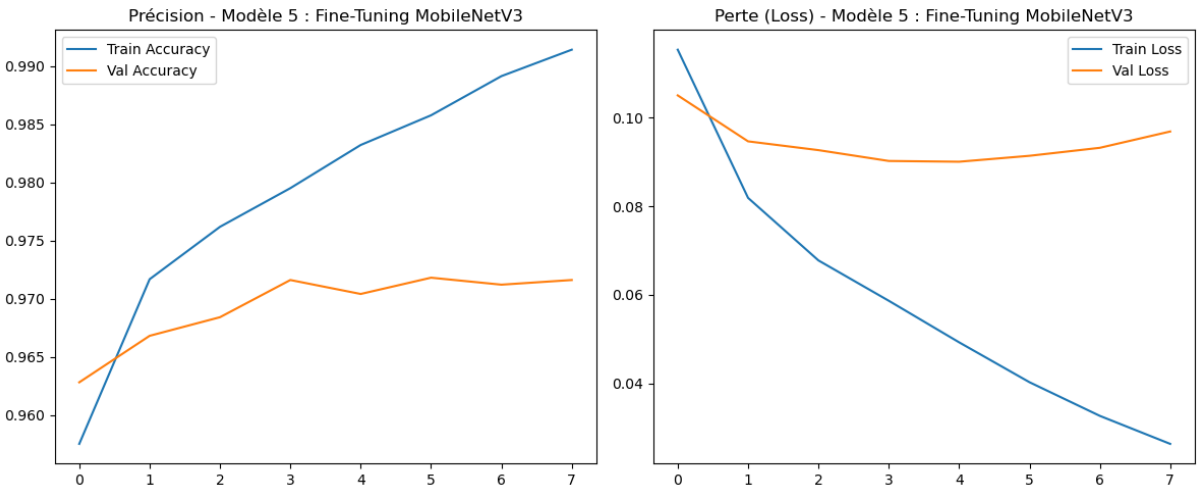


FIGURE 8 – Performance du Fine-tuning sur MobileNetV3.

Après un entraînement de 8 epochs de 35s, puis 8 epochs de fine-tuning de 70s chacunes, on obtient 97,18% de bonnes prédictions sur l'ensemble de validation, et 97,43% sur celui de test interne.

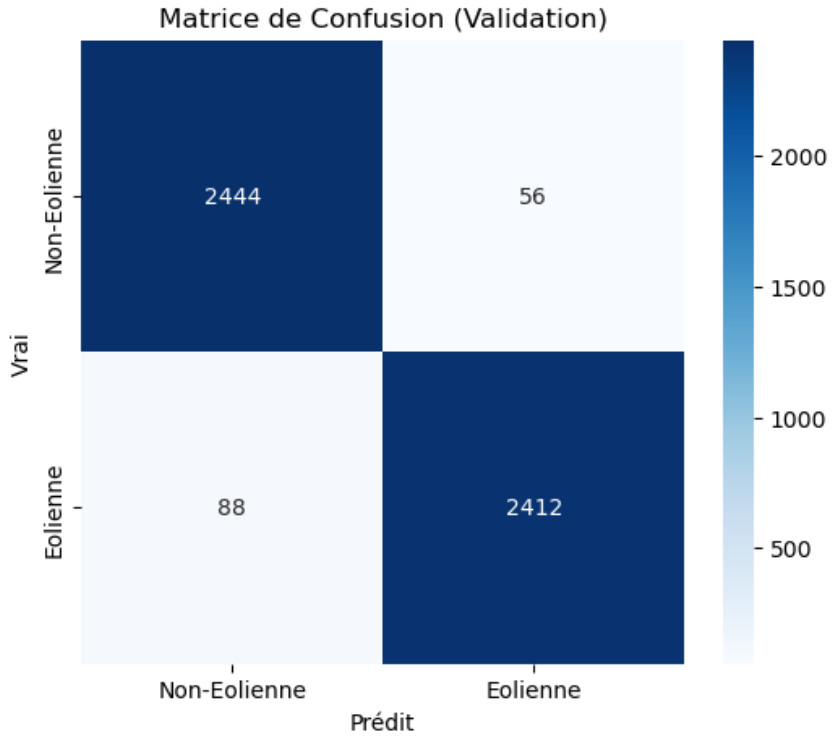


FIGURE 9 – Matrice de confusion : MobileNetV3.

4 Conclusion et Discussion

4.1 Synthèse des performances

La première remarque que l'on peut faire est que malgré l'early stopping, on peut voir sur les graphiques qu'il y a des plateaux sur la courbe validation accuracy. Cela signifie qu'il y a un petit peu de surapprentissage. Je pense que cela est dû à mon critère de stop qui nécessite 3 epochs de stagnation avant de stopper l'apprentissage.

TABLE 1 – Tableau récapitulatif des performances des modèles.

Modèle	**Nombre de paramètres**	**Précision (Val)**	**Précision (Test interne)**	**Temps / Époque**
Logistique	49 000	85,84%	85,87%	15s
CNN Simple	7 900 000	93,34%	94,10%	25s
CNN Profond	560 000	96,86%	97,28%	75s
EfficientNetV2 S	20 000 000	96,80%	97,20%	150s
MobileNetV3	3 000 000	97,18%	97,43%	70s

4.2 Analyse de la généralisation sur l'ensemble de test

En réalité, lorsque l'on passe sur les données test, les modèles stagnent autour de 90% de bonnes prédictions. Cela est assez faible comparé aux résultats sur les ensembles de validation et de test interne.

Plusieurs raisons peuvent expliquer cela :

Il est possible que certaines images soient mal étiquettées. Ou que la qualité de la photo rende l'eolienne très difficilement perceptible, même en vérifiant à la main.



FIGURE 10 – Images dont la prédiction (Modèle profond) ne correspond pas à l'étiquette.

On peut également supposer que les images de l'ensemble de test diffèrent plus des images du jeu d'entraînement que celles de validation ou de test interne (dans le cas où les photos auraient été prisent à la suite, et sans mélange).

Deuxième partie

Projet de classification de textes

5 Introduction

L'objectif de ce projet est d'expérimenter différentes méthodes de classification multi-classes de textes, et de les comparer. Pour ce faire nous disposons d'un jeu de données, provenant du site CommonCrawl, de plus de 200 000 descriptions de poste, ainsi que le nom du métier et le genre de la personne qui l'occupe.Nous possédons deux fichiers : un fichier JSON contenant les textes bruts (les descriptions de poste) et un fichier CSV contenant la catégorie cible associée à chaque identifiant. Après avoir fusionné ces données, nous obtenons un jeu d'entraînement global.



FIGURE 11 – Les 150 mots les plus fréquents (Global)

5.1 Nettoyage

Avant de donner les données à nos modèles, le texte brut nécessite un prétraitement. J'ai donc nettoyé l'ensemble des descriptions : je passe tout le texte en minuscules, je

supprime l'intégralité de la ponctuation et des caractères spéciaux, et j'élimine les espaces superflus.

De plus, on effectue une lemmatisation et un filtrage des stop words.

6 Présentation des données et statistiques descriptives

6.1 Statistiques descriptives

La principale remarque que l'on peut faire sur ce jeu de données est le fort déséquilibre entre les classes. En effet, sur les 28 catégories de métiers détectées, certaines sont surreprésentées par rapport à d'autres.

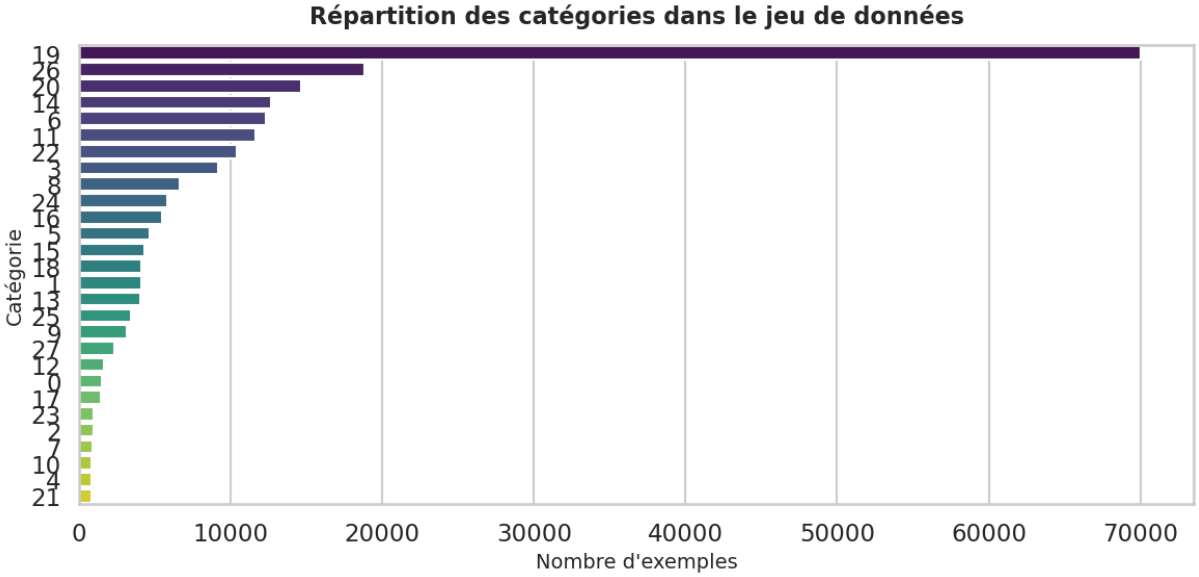


FIGURE 12 – Répartition des catégories dans le jeu de données

On remarque également que, une fois la lemmatisation effectuée, la majorité des textes ont entre 15 et 50 mots.

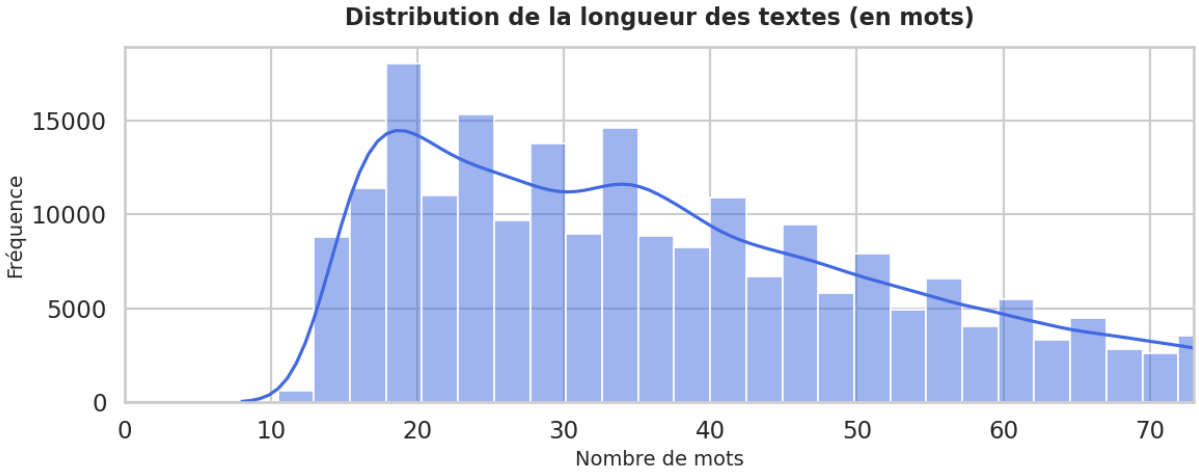


FIGURE 13 – Distribution de la longueur des textes (en mots)

6.2 Validation croisée

Pour évaluer la précision de mes modèles, j'utilise une méthode de validation croisée avec 5 itérations. Étant donné le fort déséquilibre de notre jeu de données, la simple "précision" peut parfois être trompeuse, c'est pourquoi j'ai également prêté attention aux matrices de confusion pour vérifier si les classes minoritaires étaient ignorées ou non par les modèles.

Le principe est de diviser le jeu de données en 5 blocs : à chaque itération, le modèle s'entraîne sur 4 blocs et est évalué sur le bloc restant. Cette rotation est répétée 5 fois, permettant ainsi de tester la robustesse du modèle sur l'intégralité des données et de s'assurer que la précision obtenue n'est pas le fruit d'un découpage aléatoire favorable.

7 Présentation des méthodes et premiers résultats

7.1 Modèle bayésien

Le premier modèle utilisé est une approche classique "Bag of Words", couplé à un modèle de classification naïve bayésienne (MultinomialNB). On se limite à un vocabulaire de 10000 mots. Ce premier modèle étant très simple, il est extrêmement rapide à entraîner. On obtient avec celui-ci une précision globale de 71,13% sur les 5 folds de validation croisée. C'est un bon début, mais nous voulons faire plus !

Le second modèle utilise la méthode TF-IDF afin de donner plus d'importance aux mots spécifiques d'un métier. Avec le même classifieur, et toujours un vocabulaire limité à 10000 mots, la précision est à 69,76%.

Dans les deux cas les classes peu présentes sont correctement prédites, c'est le cas avec

les classes qui apparaissent environt 800 fois comme les classes 7, 10 et 21 avec plus de 99% de précision. En revanche, les classes plus présentes sont moins bien détectés. C'est le cas de la classe 3, présente 9000 fois, avec un score de seulement 40%. La classe 19, qui est la plus présente avec plus de 70000 textes n'est qu'à 72%.

On obtient a peu près la même matrice de confusion, sur laquelle on peut voir la diagonale des prédictions correctes.

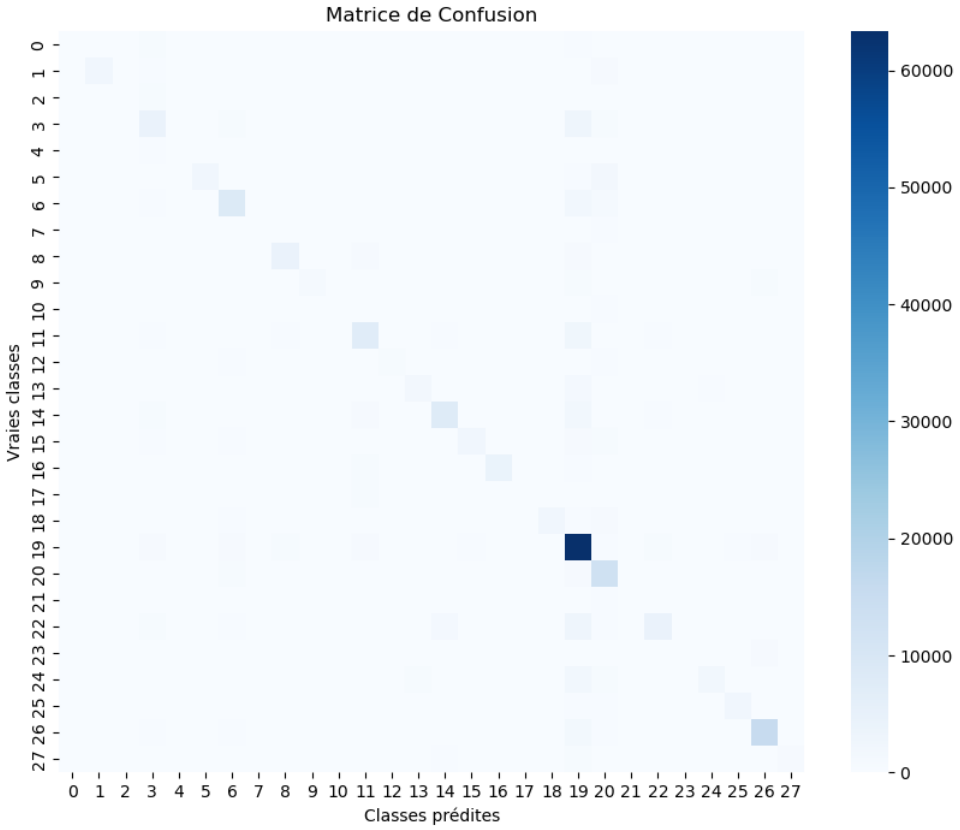


FIGURE 14 – Matrice de confusion sur tf-idf (semblable à Bow)

7.2 Réseau dense

Pour pouvoir passer à l'utilisation de réseau de neurones dense, il est nécessaire de changer de stratégie car limiter le vocabulaire ne suffit plus, la taille des matrices creuses obtenus est bien trop coûteux. L'astuce que j'ai mis en place est de faire une SVD (décomposition en éléments simples). Le modèle s'entraîne désormais sur des représentations denses beaucoup plus compactes (300 dimensions).

Cela nous donne alors un modèle à un peu plus de 100 000 paramètres à entrainer, ce qui ne prends qu'une dizaine de seconde par epoch.

On obtient alors une précision de 76,00% avec le BOW, et 77,29% avec le tf-idf.

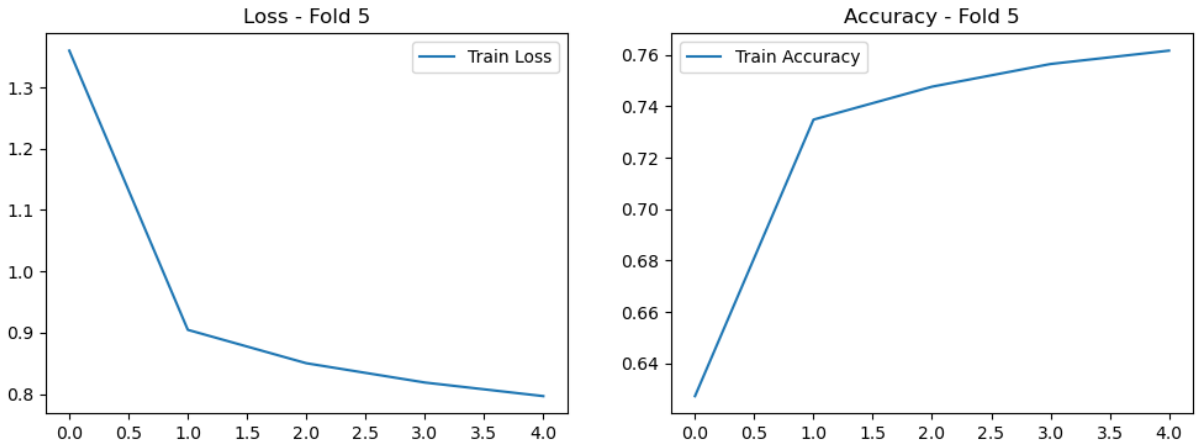


FIGURE 15 – Courbes entraînement du modèle dense tf-idf (semblable à Bow)

7.3 LSTM

Améliorons ces résultats en passant sur une architecture plus adaptée au texte. J'ai utilisé une couche d'Embedding suivie d'un réseau récurrent de type LSTM à 64 unités, avec du Dropout. Pour ce faire, j'ai uniformisé la longueur de mes séquences de texte à l'aide d'un padding. J'obtiens alors des vecteurs de taille 100, ce qui fait un modèle à 1 000 000 de paramètres entrainables. Que ce soit avec des embeddings aléatoire ou de word2Vec, la précision est de seulement 32,25%. En regardant la matrice de confusion on comprends ce qu'il se passe : la classe 19 constituant à elle seul un tier de notre jeu de données, le modèle a choisi de ne prédire plus que celle-ci.

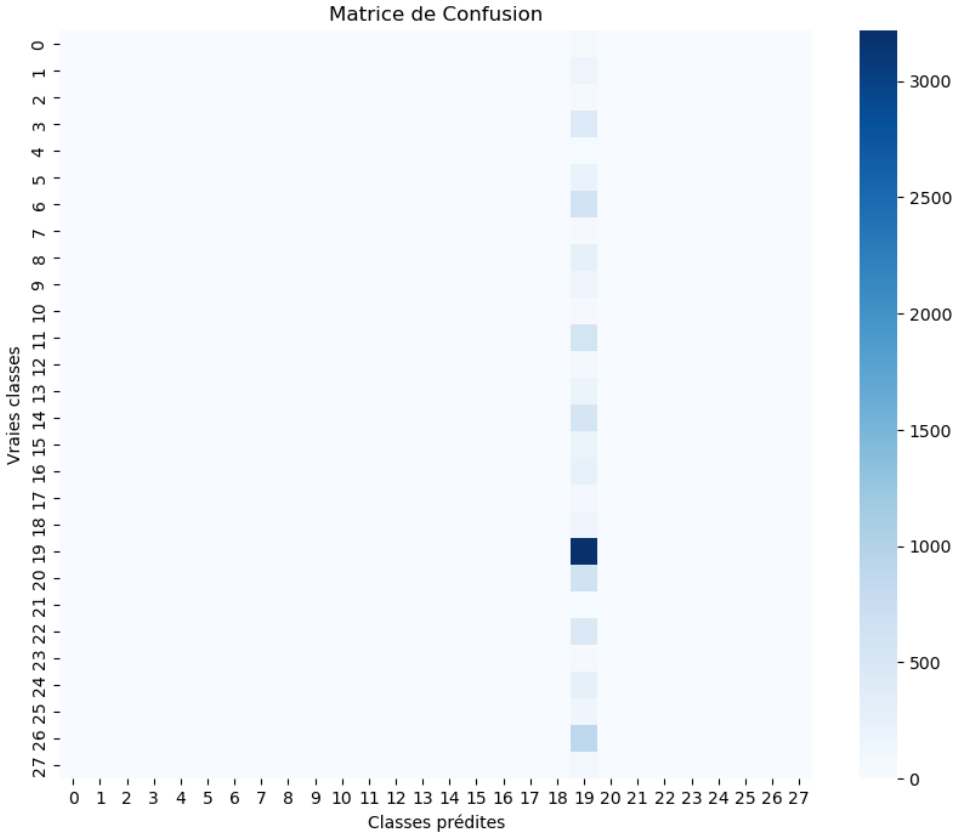


FIGURE 16 – Matrice de confusion LSTM

8 Gestion des poids

Comme nous venons de le voir, l'hétérogénéité de la répartition des catégories peut poser problème. Pour résoudre celui-ci, une solution est de mettre en place des poids : si une catégorie est peu représentée, se tromper sur celle-ci va beaucoup influer sur la fonction de perte, et inversement.

Après application des poids les modèles bayésiens et réseaux denses restent inchangés. En revanche, pour ce qui est des LSTM avec embeddings il y a des changements. Tout d'abord l'entraînement nécessite une quinzaine d'epochs de 30s pour converger. De plus, la précision s'est nettement améliorée car cette fois ci le modèle avec embeddings aléatoires obtient une précision de 76.27%, et 74,45% avec les embeddings word2Vec.

Cette fois-ci ce sont les classes les moins présentes qui sont le moins bien prédites.

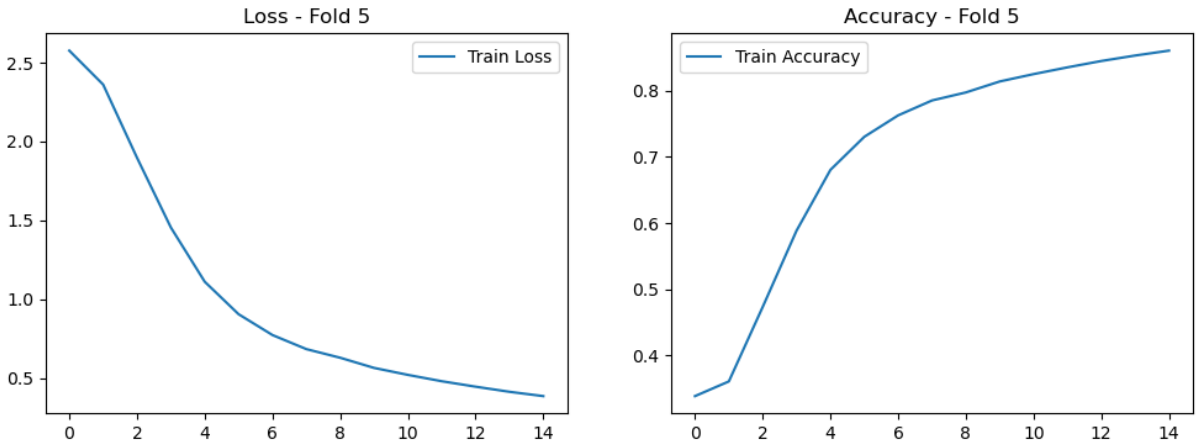


FIGURE 17 – Courbe entraînement LSTM embeddings aléatoires

Bien que l'ajout des poids de classes ait permis de débloquer l'apprentissage du LSTM et d'éviter qu'il ne prédise que la classe majoritaire, on remarque que les performances globales ne décollent pas autant qu'espéré. Paradoxalement, le réseau Dense couplé à la réduction de dimension (SVD) sur le TF-IDF reste légèrement plus performant et surtout beaucoup plus rapide à entraîner.

9 Conclusion et discussion

En analysant le tableau récapitulatif ci-dessous, plusieurs constats s'imposent. Tout d'abord, c'est l'approche TF-IDF couplée à un réseau de neurones dense qui offre le meilleur compromis sur ce projet. Avec une précision de 77,29% et un temps d'entraînement très court (10s par epoch), ce modèle surpasse des architectures plus complexes.

À l'inverse, les modèles récurrents (LSTM), bien que conçus pour capturer le contexte séquentiel du texte, sont beaucoup plus lourds et peinent à dépasser ce score. Même en combinant des embeddings et une gestion stricte des poids pour contrer le déséquilibre des classes, l'architecture LSTM semble atteindre ses limites sur ces données.

Nous semblons avoir atteint un "plafond de verre" autour de 76-77% de précision globale. Plusieurs facteurs peuvent expliquer cette stagnation : d'une part, le vocabulaire de certains métiers très proches (comme *Software Engineer* et *Data Scientist*) se recoupe énormément, rendant la tâche de classification difficile avec des plongements de mots classiques ; d'autre part, la réduction de dimension et le nettoyage font parfois perdre de précieuses nuances de langage.

Pistes d'amélioration (Transformers)
Pour aller plus loin et franchir ce palier technique, la suite logique de ce projet serait de délaisser les réseaux récurrents au profit de l'architecture des Transformers. L'utilisation

de modèles de langage massifs et pré-entraînés tels que BERT, via une technique de fine-tuning, apporterait une réelle plus-value. Cependant les contraintes techniques qu'apporte ce modèle m'ont contraint à ne pas pouvoir l'expérimenter.

TABLE 2 – Tableau récapitulatif des performances des modèles.

Modèle	**Paramètres**	**Précision**	**Poids**	**Epochs**	**Temps / epoch**
BOW Bayésien	280 000	71,13%	Non	5	3s
TF-IDF Bayésien	280 000	69,76%	Non	5	3s
BOW Dense	100 000	76,00%	Non	5	10s
TF-IDF Dense	100 000	77,29%	Non	5	10s
LSTM Aléatoire	1 000 000	76,27%	Oui	15	30s
LSTM Word2Vec	1 000 000	74,45%	Oui	15	30s